Microprocessor and Assembly Language

CSE 0611-3321

Representation of Numbers and Characters

Presented By

Saifur Rahman

Lecturer

Dept. of CSE

NBIU

References

Assembly Language Programming and Organization of the IBM PC

By Ytha Yu and Charles Maru

Number Systems

- A **number system** is a way to represent and express numbers using a set of symbols and rules.
- In a positional system, a base ($\boldsymbol{b}$) is chosen. Symbols are assigned to numbers between ***0*** and ***b – 1.***
- Different number systems use different bases to represent numbers. Common types include:
  - Decimal system (Base 10) : Uses digits 0-9.
  - Binary system (Base 2) : Uses digits 0 and 1, commonly used in computers.
  - Octal system (Base 8) : Uses digits 0-7.
  - Hexadecimal system (Base 16) : Uses digits 0-9 and letters A-F to represent values 1015.

Each digit's position in a number determines its value based on the base of the system.

Number Systems (cont'd)

Representation

- Decimal system (Base 10) : **65, 65d,** $(\mathbf{65})_{\mathbf{10}}$
- Binary system (Base 2) : **0b1000101,** $(\mathbf{1000101})_{\mathbf{2}}$
- Octal system (Base 8) : **0o72,** $(\mathbf{72})_{\mathbf{8}}$
- Hexadecimal system (Base 16) : **0x7A,** $(\mathbf{7A})_{\mathbf{16}}$

Decimal Number System

- A positional number system assigns each digit a value based on its position and a specific base (such as 10 in the decimal system).
- For example:

The number 3932 represents:

$$3932 = 3 * 10^3 + 9 * 10^2 + 3 * 10^1 + 2 * 10^0$$

- 3 thousands (3 × 1000)
- 9 hundreds (9 × 100)
- 3 tens (3 × 10)
- 2 ones (2 × 1)

Binary Number System

In the binary number system:

- Base is 2, with only two digits: ***0*** and ***1***.
- Example: The binary string ***11010*** equals:

$$11010 = 1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 = 26$$

$$\mathbf{(11010)_2 = (26)_{10}}$$

- Base 2 is represented in binary as 10.

Hexadecimal Number System

In assembly language programming, binary numbers can be long and hard to work with. For example:

- 16 bits are needed to represent a memory word in an 8086based computer.
- Decimal numbers are difficult to convert into binary.

To make it easier, we often use the hexadecimal (hex) system:

- Hex is a base16 number system with digits: ***09*** and ***AF (A = 10, F = 15)***.
- After ***F***, base ***16*** is represented as ***10***.
- **Each hex digit corresponds to 4 bits**, making it easy to convert between binary and hex.

Hexadecimal Number System (cont'd)

Table 1: Hex Digits and Binary Equivalent

Hex Digits	Binary Digits
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Hex Digits	Binary Digits
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111
10	0001 0000
11	0001 0001
12	0001 0010
…	…

Decimal, Binary, and Hex Numbers

Table 2: Decimal, Binary, and Hex Digits

Decimal	Hex	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001

Decimal	Hex	Binary
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111
16	10	0001 0000
…	…	…
256	100	1 0000 0000
…	…	…

Converting Binary & Hex to Decimal

Consider the hex number ***8A2Dh***. It can be written as:

$$\boldsymbol{8A2Dh = 8 * 16^3 + A * 16^2 + 2 * 16^1 + D * 16^0}$$

$$\boldsymbol{= 8 * 16^3 + 10 * 16^2 + 2 * 16^1 + 13 * 16^0 = 35373\mathrm{d}}$$

Similarly, the binary number ***11101b*** may be written as

$$\boldsymbol{11101b = 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 = 29d}$$

Converting Binary & Hex to Decimal (cont'd)

Position	**…**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Power of 2 (base)	…	$\mathbf{2^8}$	$\mathbf{2^7}$	$\mathbf{2^6}$	$\mathbf{2^5}$	$\mathbf{2^4}$	$\mathbf{2^3}$	$\mathbf{2^2}$	$\mathbf{2^1}$	$\mathbf{2^0}$
	…	256	128	64	32	16	8	4	2	1
Bits	…									

If a bit is **1**, add **2 raised to the position of the bit** (starting from 0).
Add all these values together to get the decimal number.

Converting Binary & Hex to Decimal (cont'd)

Convert ***1010b*** to decimal.

Position	**…**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Power of 2 (base)	…	$\mathbf{2^8}$	$\mathbf{2^7}$	$\mathbf{2^6}$	$\mathbf{2^5}$	$\mathbf{2^4}$	$\mathbf{2^3}$	$\mathbf{2^2}$	$\mathbf{2^1}$	$\mathbf{2^0}$
	…	256	128	64	32	16	8	4	2	1
Bits	…						1	0	1	0
Sum = 8 + 2 = ***10d***										

- ***10d***

Converting Binary & Hex to Decimal (cont'd)

Convert ***1000101b*** to decimal.

Position	**…**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Power of 2 (base)	…	$\mathbf{2^8}$	$\mathbf{2^7}$	$\mathbf{2^6}$	$\mathbf{2^5}$	$\mathbf{2^4}$	$\mathbf{2^3}$	$\mathbf{2^2}$	$\mathbf{2^1}$	$\mathbf{2^0}$
	…	256	128	64	32	16	8	4	2	1
Bits	…			1	0	0	0	1	0	1
Sum = 64 + 4 + 1 = ***69d***										

- ***69d***

Converting Binary & Hex to Decimal (cont'd)

➢Convert ***11101b*** to decimal.

Bits	Digit Base + Next Digit
1	
1	$1 * 2 + 1 \rightarrow 3$
1	$3 * 2 + 1 \rightarrow 7$
0	$7 * 2 + 0 \rightarrow 14$
1	$14 * 2 + 1 = \mathbf{29}$

1 1 1 0 1

$1 * 2 + 1 \rightarrow 3 * 2 + 1 \rightarrow 7 * 2 + 0 \rightarrow 14 * 2 + 1 = \mathbf{29d}$

Converting Binary & Hex to Decimal (cont'd)

➢Convert ***2BD4h*** to decimal.

2 B D 4

$$2 * 16 + 11 \rightarrow 43 * 16 + 13 \rightarrow 701 * 16 + 04 = \mathbf{11220d}$$

Class Work: What is the decimal of ***DEADBEEFh ?***

Answer: 3735928559d

Converting Decimal to Binary & Hex

1. **Divide** the number by **base**.
2. **Write down** the remainder.
3. **Divide** the result again by **base** and write down the remainder.
4. **Repeat** until the quotient is 0.
5. **Combine** the remainders in reverse order for the result.

Converting Decimal to Binary & Hex

➢Convert ***11172*** to hex.

To convert 11172:

- $11172 \div 16 = 698$, remainder 4
- $698 \div 16 = 43$, remainder A
- $43 \div 16 = 2$, remainder B
- $2 \div 16 = 0$, remainder 2

So, ***11172*** is ***2BA4*** in hexadecimal.

$$11172 = 698 * 16 + 4$$
$$698 = 43 * 16 + 10\ (Ah)$$
$$43 = 2 * 16 + 11\ (Bh)$$
$$2 = 0 * 16 + 2$$

Remainder in Reverse order $2BA4h$

Class Work: Convert ***1234 to hex.***

Answer: 4D2h

Converting Decimal to Binary & Hex

➢Convert ***95*** to binary.

$$95 = 47 * 2 + 1$$
$$47 = 23 * 2 + 1$$
$$23 = 11 * 2 + 1$$
$$11 = 5 * 2 + 1$$
$$5 = 2 * 2 + 1$$
$$2 = 1 * 2 + 0$$
$$1 = 0 * 2 + 1$$

Remainder in Reverse order

$1011111b$

Conversion Between Binary & Hex

To convert a **hex number to binary**:

- **Change each hex digit** to its 4bit binary equivalent.

- Convert 2B3Ch to binary.

2	B	3	C
0010	1011	0011	1100

0010 1011 0011 1100

Conversion Between Binary & Hex (cont'd)

To convert **binary to hex**:

- **Group the binary digits** into sets of four, starting from the right.
- **Convert each group** to a hex digit.

- Convert 1110101010 to hex.

11	1010	1010
0011	1010	1010
3	A	A

3AAh

Important Bits

Two important bits are:

- **Most significant bit (msb):** The leftmost bit (bit 15 in a word, bit 7 in a byte).
- **Least significant bit (lsb):** The rightmost bit (bit 0).

Position	7	6	5	4	3	2	1	0
Contents	**MSB**							**LSB**

Integer Representation

Computers represent integers in binary (0s and 1s). The way these integers are stored depends on whether they are **unsigned** or **signed**.

- Unsigned Integers
  - Represent **nonnegative numbers** (no negative values).
  - Used for things like **memory addresses**, **counters**, and **ASCII codes**.
  - All bits are used to represent the number, with no need for a sign bit.
  - An **8bit byte** can store values from **0 to 255** (**`11111111` = `FFh` = `255`**).
  - A **16bit word** can store values from **0 to 65,535** (**`1111111111111111` = `FFFFh` = `65,535`**).
  - If the least significant bit (lsb) is **1**, the number is **odd**; if it's **0**, the number is **even**.

Integer Representation (cont'd)

- Signed Integers
  - Can be either **positive** or **negative**.
  - The **most significant bit (msb)** is used for the **sign**:
    - **1** means **negative**.
    - **0** means **positive**.
  - **Negative** integers are stored using **two's complement**.
  - **One's complement** is the first step in understanding **two's complement**.

Integer Representation (cont'd)

- One's Compliment
  - One's complement is created by flipping all bits of a number.
  - Change each 0 to 1 and each 1 to 0.

❖Find the One's Compliment of 5.

5 =	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**1**
1's Comp.	1	1	1	1	1	0	1	0

Sign Bit (8 bits)

Integer Representation (cont'd)

- Two's Compliment
  - First, get the **one's complement** (flip the bits).
  - Then, **add 1** to the result.

❖Find the Two's Compliment of 5.

5 =	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**1**
1's Comp.	1	1	1	1	1	0	1	0
							+	1
2's Comp.	1	1	1	1	1	0	1	1

Sign Bit (8 bits)

Integer Representation (cont'd)

- **2's compliment of 5 is 5 and 2's Comp. of 5 is 5.**

5 =	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**1**
-5 =	1	1	1	1	1	0	1	1
1's Comp.	0	0	0	0	0	1	0	0
							+	1
2's Comp.	0	0	0	0	0	1	0	1

Decimal Interpretation

Unsigned Decimal Interpretation

- Convert the binary directly to decimal.

Signed Decimal Interpretation

- If the most significant bit (msb) is **0**, the number is **positive**, and the signed decimal is the **same as the unsigned**.
- If the msb is **1**, the number is **negative**. To find the value:
  - Take the **two's complement** of the number.
  - Convert the result to decimal.

Decimal Interpretation (cont'd)

Table 3: Signed. and Unsigned Decimal Interpretations of 16Bit Register/ Memory Content

Hex	Unsigned Decimal	Signed Decimal
0000	0	0
…	…	…
0009	9	9
000A	10	10
…	…	…
7FFE	32766	32766
7FFF	32767	32767
8000	32768	-32768
8001	32769	-32767
…	…	…
FFFE	65534	-2
FFFF	65535	-1

ASCII Code

- **ASCII (American Standard Code for Information Interchange)** is the main character encoding system.
- ASCII uses **7 bits per character**, allowing for **128 unique codes**.
- **Printable ASCII codes** range from 32 to 126.
  - Examples: 32 (20h): **Space**, 33 (21h): **Exclamation mark (!)**, 48 (30h): **Digit 0**
- Codes 0 to 31 and 127 are **control characters**, not printable.
  - Examples: 0 (00h): **Null (NUL)**, 7 (07h): **Bell (BEL)**, 8 (08h): **Backspace (BS)** etc.
- Each ASCII character fits into a **byte** (8 bits), with the most significant bit usually set to zero.
- **Printable characters** appear on screens or printers, while **control characters** manage device operations.
- Some systems use an **extended ASCII** set with 256 characters for more display options.

ASCII Code (cont'd)

dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	`
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	-	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	.	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	{
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	|
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	}
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	~
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

ASCII Code (cont'd)

❖How the character string "RG 2z" is stored in memory, starting address 0.

From ASCII Table,

Character	ASCII Code (Hex)	ASCII Code (Binary)
R	52	0101 0010
G	47	0100 0111
space	20	0010 0000
2	32	0011 0010
z	7A	0111 1010

So, Memory:

Address	Contents
0	01010010
1	01000111
2	00100000
3	00110010
4	01111010